

Data Spaces and Federated Data Engineering for Orbital and Space Observations

Laboratory 4: Querying and Integrating Data in a Data Space

Leszek Siwik
Faculty of Space Technologies
AGH University of Krakow

1 Introduction

In previous laboratories, you processed datasets using shell scripts.

While this approach allows automation, it does not reflect how data is accessed in modern data systems.

Instead of manually scanning files, queries describe **what data is needed**, and the system determines **how to retrieve it**.

In this laboratory, you will use DuckDB to query distributed datasets directly from CSV files, and simulate data space and federated access patterns.

During this laboratory, you will learn how to:

- query CSV datasets using SQL,
- filter and aggregate data,
- query multiple providers,
- identify data origin,
- simulate federated queries,
- validate schema consistency,
- interpret results in a data space context.

2 Required Input Data

Use the same OODS workspace as in previous laboratory.

```
oods-system/  
  providers/  
    satellite_A/  
      observations.csv  
    satellite_B/  
      observations.csv  
    ground_station/  
      observations.csv  
  scripts/  
  reports/
```

3 Task 1: Running DuckDB

Start DuckDB in the terminal:

```
duckdb
```

If duckdb command is not recognized / not installed do follow the hint explaining how to install it:

```
sudo snap install duckdb
```

After successful installation and recalling duckdb you should get:

```
/oods-system-lab-3$ duckdb
v1.1.3 19864453f7
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
D
```

Important: When writing SQL queries, make sure to use standard single quotes:

```
'providers/satellite_A/observations.csv'
```

Do not use typographic quotes, as they will cause syntax errors in DuckDB.

4 Task 2: First steps

DuckDB allows querying CSV files directly without importing them like:

```
SELECT * FROM 'providers/satellite_A/observations.csv';
```

And you should get something similar to:

timestamp	object_id	temperature	velocity
2026-03-01 ...	OBJ-001	15.1	7.4
2026-03-01 ...	OBJ-002	14.9	7.2
...			

In addition to displaying data, it is often necessary to understand the structure of a dataset. DuckDB allows inspecting the schema using the DESCRIBE command like:

```
DESCRIBE SELECT * FROM 'providers/satellite_A/observations.csv';
```

What returns column names and their data types. It is useful for verifying whether datasets share the same structure what is essential when integrating data from multiple providers.

And generally, depending on the DuckDB version / mode, you should get something similar to:

column_name	column_type	null	key	default	extra
timestamp	TIMESTAMP	YES			
object_id	VARCHAR	YES			
temperature	DOUBLE	YES			
velocity	DOUBLE	YES			

Task

For practicing, execute queries that:

- inspect schemas of `satellite_A` and `satellite_B`,
- compare column names and data types,

- determine whether the schemas are identical or not.

And to avoid any misunderstandings try to get something similar to:

Schema comparison:

```
satellite_A:
timestamp    TIMESTAMP
object_id    VARCHAR
temperature  DOUBLE
velocity     DOUBLE
```

```
satellite_B:
timestamp    TIMESTAMP
object_id    VARCHAR
temperature  DOUBLE
velocity     DOUBLE
```

RESULT: schemas match

5 Task 2: Filtering Observations for Analysis

In a dataset, multiple observations may refer to the same object. At the same time in a data spaces, analysis is often object-centric.

Filtering allows isolating a subset of data that is relevant for further analysis, as shown below:

```
SELECT *
FROM 'providers/satellite_A/observations.csv'
WHERE object_id = 'OBJ-003';
```

Or for instance to count the occurrences of the specific object we may use where along with count like below:

```
SELECT COUNT(*)
FROM 'providers/satellite_A/observations.csv'
WHERE object_id = 'OBJ-003';
```

And depending on the current state of the system we should get something similar to:

```
COUNT
1
```

Task

For practicing, preate queries that:

- filter observations for a selected object (e.g. OBJ-003) in `satellite_A`,
- count how many observations exist for this object in that dataset,
- repeat the same analysis for `satellite_B`,
- compare the results between providers,
- determine whether the object is equally represented in both datasets.

And again, to avoid misunderstandings, try to get something similar to:

OBJ-003 occurrences:

```
satellite_A: 1
satellite_B: 1
```

COMPARISON: equal representation

6 Task 3: Querying Multiple Providers

In a data spaces, datasets are distributed across multiple providers. So a single object may be observed independently by different sources.

To analyze such data, it is necessary to combine datasets into a unified view, and we may do that as below:

```
SELECT *  
FROM 'providers/*/observations.csv';
```

Where the wildcard * matches all provider directories, so the result contains observations from multiple providers as all datasets are read together as one logical table

We may do the same e.g to count all the occurrences of the objects across all datasets like:

```
SELECT COUNT(*)  
FROM 'providers/*/observations.csv';
```

and we should get something like:

```
COUNT  
15
```

Task

So, for practicing, create queries that:

- count the number of observations in `satellite_A`,
- count the number of observations across all providers,
- compare both values,
- identify objects that appear more than once in the combined dataset,
- determine whether combining datasets reveals additional information about objects.

Expected Output

Observations:

satellite_A: 6

ALL PROVIDERS: 15

Objects appearing multiple times:

OBJ-001

OBJ-003

CONCLUSION:

Combined query reveals additional observations not visible in a single dataset.

7 Task 4: Identifying Data Origin

When querying multiple provider datasets at the same time, the result no longer comes from a single source.

In such cases, it is necessary to identify which provider produced each observation. Without this information, the result is only a combined table. With source information, it becomes possible to interpret the data correctly. This can be achieved in DuckDB as follows:

```
SELECT filename, *
FROM read_csv_auto('providers/*/observations.csv', filename=true);
```

where

- the pattern `providers/*/observations.csv` matches all provider datasets,
- `read_csv_auto(...)` reads them as a single logical table,
- `filename=true` adds an additional column with the source file path,
- each row is now associated with the provider that produced it.

This approach can be combined with filtering, for example:

```
SELECT filename, *
FROM read_csv_auto('providers/*/observations.csv', filename=true)
WHERE object_id = 'OBJ-003';
```

The result should be similar to:

filename	timestamp	object_id	temperature
velocity			
providers/satellite_A/observations.csv	2026-03-01 12:10:00	OBJ-003	15.0
providers/satellite_B/observations.csv	2026-03-01 12:12:00	OBJ-003	14.8
providers/ground_station/observations.csv	2026-03-01 12:11:00	OBJ-003	15.2

From this result, we can observe:

- whether the same object appears in multiple providers,
- how many providers contribute observations,
- whether values differ between providers,
- how adding the filename changes interpretation.

Task

For practicing, create queries that:

- display all observations of `OBJ-003` together with their source,
- determine in which providers `OBJ-003` appears,
- count how many observations come from each provider,
- compute the total number of observations across all providers.

Hint: the provider name can be extracted from the file path stored in the `filename` column.

Expected Output

The following is a simplified example of the expected result of the analysis:

```
OBJ-003 appears in:
satellite_A
satellite_B
ground_station
```

```
Per-provider counts:
satellite_A: 1
satellite_B: 1
ground_station: 1
```

TOTAL: 3

8 Task 5: Aggregation Across Providers

When data is distributed across multiple providers, it is often necessary to summarize and compare datasets.

Aggregation allows transforming raw observations into higher-level information, such as counts, totals, or rankings.

In a data space context, aggregation is used to understand:

- how much data each provider contributes,
- whether the data distribution is balanced,
- which provider dominates the data space.

In the following example:

```
SELECT filename, COUNT(*)
FROM read_csv_auto('providers/*/observations.csv', filename=true)
GROUP BY filename;
```

- GROUP BY filename groups observations by source file (provider dataset),
- COUNT(*) counts the number of rows in each group,
- the result shows how many observations come from each provider dataset.

We can also order the results to highlight the most relevant information:

```
SELECT filename, COUNT(*) AS observations
FROM read_csv_auto('providers/*/observations.csv', filename=true)
GROUP BY filename
ORDER BY observations DESC;
```

The result should be similar to:

filename	observations
providers/satellite_A/observations.csv	6
providers/satellite_B/observations.csv	5
providers/ground_station/observations.csv	4

This makes it easier to observe:

- which provider has the largest number of observations,
- which provider has the smallest dataset,
- how large the difference between providers is,
- whether the data space appears balanced or skewed.

Task

Create queries that:

- count the number of observations in each provider dataset,
- order providers from largest to smallest dataset,
- identify which provider has the largest number of observations,
- identify which provider has the smallest number of observations,
- compute the total number of observations across all providers (for verification).

Hint: the provider name can be extracted from the file path stored in the filename column.

Expected Output

```
Provider ranking (by number of observations):  
satellite_A: 6  
satellite_B: 5  
ground_station: 4
```

```
Largest dataset: satellite_A  
Smallest dataset: ground_station
```

```
TOTAL OBSERVATIONS: 15
```

9 Task 6: Full Data Space Query

Until now, you have worked with individual datasets and simple aggregations.

In a data space, the key question is not only how data is structured, but also how information about a single object is distributed across providers.

A full data space query attempts to reconstruct a complete view of an object by combining all available observations, as shown below:

```
SELECT filename, timestamp, object_id, temperature, velocity  
FROM read_csv_auto('providers/*/observations.csv', filename=true)  
WHERE object_id = 'OBJ-003';
```

In this query:

- the query scans all provider datasets,
- it filters observations for a single object,
- it returns all matching rows together with their source,
- this creates a unified, cross-provider view of the object.

We can also count observations across the data space, for example:

```
SELECT COUNT(*) AS total_observations  
FROM read_csv_auto('providers/*/observations.csv')  
WHERE object_id = 'OBJ-003';
```

The result should be similar to:

```
total_observations  
3
```

Task

Create queries that:

- retrieve all observations of a selected object (e.g. OBJ-003) across all providers,
- list all providers that contain this object,
- count how many observations are recorded for this object in total,
- determine whether any provider is missing this object.

Assume that the complete set of providers is known (e.g. satellite_A, satellite_B, ground_station).

Hint: the provider name can be extracted from the file path stored in the `filename` column.

Expected Output

The following is a simplified representation of the result:

OBJ-003 observations:

```
providers/satellite_A/... 2026-03-01 ... 15.0 7.5
providers/satellite_B/... 2026-03-01 ... 14.8 7.3
providers/ground_station/... 2026-03-01 ... 15.2 7.6
```

Providers containing OBJ-003:

```
satellite_A
satellite_B
ground_station
```

TOTAL OBSERVATIONS: 3

Missing providers: none

10 Task 7: Federated Query Simulation

In the previous task, you executed a full data space query, which assumed that all provider datasets are available.

In real systems, however, access may be limited:

- some providers may not be accessible,
- some datasets may not be registered in metadata,
- some queries may intentionally use only selected sources.

This leads to the concept of federated queries, where only a subset of the data space is queried. Consider the following examples:

Step 1: Full Data Space Query

```
SELECT COUNT(*) AS total
FROM read_csv_auto('providers/*/observations.csv')
WHERE object_id = 'OBJ-003';
```

In this query:

- all provider datasets are included,
- the result represents the complete view of the data space,
- this serves as a reference (complete result).

Step 2: Federated Query (Partial Access)

```
SELECT COUNT(*) AS total
FROM read_csv_auto(
  ['providers/satellite_A/observations.csv',
   'providers/satellite_B/observations.csv']
)
WHERE object_id = 'OBJ-003';
```

In this query:

- only selected providers are queried,
- the list of files explicitly defines which providers are included,
- one provider (e.g. ground_station) is excluded,
- the result may no longer represent the full data space.

The results may be similar to:

```
FULL RESULT: 3
FEDERATED RESULT: 2
```

Task

Create queries that:

- execute a full data space query for a selected object,
- execute a federated query using only a subset of providers,
- compare both results,
- identify which providers are missing,
- determine whether the federated result is complete or incomplete.

Assume that the complete set of providers is known (e.g. `satellite_A`, `satellite_B`, `ground_station`).

Expected Output

```
FULL RESULT: 3
FEDERATED RESULT: 2

MISSING PROVIDERS:
ground_station

COMPLETE: NO (federated result differs from full data space result)
```

This demonstrates that federated queries may produce incomplete results if not all providers are included.

11 Task 8: Schema Consistency Validation

In previous tasks, you combined datasets from multiple providers into a single query.

This operation assumes that all datasets share a compatible structure.

In practice, this assumption is often violated, as:

- column names may differ,
- data types may not match,
- some providers may include additional fields,
- some providers may be missing expected fields.

Before integrating data, it is therefore necessary to verify whether schemas are consistent.

In DuckDB, this can be done using the `DESCRIBE` statement:

```
DESCRIBE SELECT * FROM 'providers/satellite_A/observations.csv';
```

```
DESCRIBE SELECT * FROM 'providers/satellite_B/observations.csv';
```

Repeat this for all available providers (e.g. `satellite_A`, `satellite_B`, `ground_station`).

- the `DESCRIBE` command returns column names and types,
- this defines the schema of each dataset,
- comparing schemas allows determining compatibility,
- compatible schemas can be safely combined in queries.

Task

Create queries that:

- inspect schemas of all provider datasets,
- compare column names and types,
- determine whether schemas are consistent across all providers,
- identify any differences (missing or extra columns, type mismatches),
- classify the data space as:
 - CONSISTENT - if all providers share the same schema,
 - INCONSISTENT - if any difference is detected.

Hint: pay attention to:

- column names,
- column order,
- data types.

Expected Output

```
Schema comparison:

satellite_A: OK
satellite_B: OK
ground_station: MISSING COLUMN velocity

RESULT: INCONSISTENT
```

Schema inconsistencies may prevent correct data integration or lead to incorrect query results.

12 Task 9: Data Space Query Report

In previous tasks, you performed individual operations:

- filtering observations,
- combining datasets,
- identifying data origin,
- aggregating results,
- comparing full and federated queries,
- validating schema consistency.

Each of these steps provides partial insight into the data space.

In real systems, however, such information must be collected, summarized, and presented in a structured form.

This leads to the concept of a **data space report**, which aggregates key information about:

- data availability,
- data distribution,
- query completeness,
- schema compatibility.

Create a script:

```
query_report.sh
```

that:

- executes DuckDB queries,
- reuses queries and logic developed in previous tasks (Tasks 3–8),

- takes the object identifier as a parameter (e.g. OBJ-003),
- uses a predefined subset of providers (e.g. satellite_A and satellite_B) for the federated query,
- generates a structured report file:

```
../reports/query_report_YYYYMMDD_HHMMSS.txt
```

Report Requirements

The report must include:

- total number of observations across all providers,
- number of distinct object identifiers,
- total number of observations for a selected object,
- list of providers that contain this object,
- result of federated vs full query comparison:
 - full result,
 - federated result,
 - completeness status,
- schema consistency status (based on Task 8).

Implementation Hint

The script may execute DuckDB queries using:

```
duckdb -c "SQL_QUERY"
```

and redirect results to a file using standard shell mechanisms.

Expected Output

```
DATA SPACE QUERY REPORT
-----
Generated at: 2026-04-14 10:30:00

[GLOBAL STATISTICS]
Total observations: 15
Distinct objects: 6

[OBJECT ANALYSIS: OBJ-003]
Providers containing object:
- satellite_A
- satellite_B
- ground_station
Total observations: 3

[FEDERATED QUERY COMPARISON]
FULL RESULT: 3
FEDERATED RESULT: 2
COMPLETE: NO

[SCHEMA VALIDATION]
Schema consistency: CONSISTENT
```

This report summarizes key aspects of the data space and provides a consolidated view of data availability and quality.

13 Practice Tasks

The following tasks are designed to simulate real-world data space challenges.

Unlike previous tasks, they require:

- combining multiple query techniques,
- interpreting results,
- identifying limitations of the data space,
- making engineering decisions based on incomplete information.

Each task should be implemented as a script using DuckDB.

Task P1: Object Presence Map

Create a script:

```
object_presence_map.sh
```

The script should:

- identify all distinct object identifiers,
- determine in which providers each object appears across the data space,
- represent the result as a simple matrix, where each row corresponds to an object and each column to a provider,
- use a clear marker such as X for presence and - for absence.

What we should get is:

OBJECT	A	B	G
OBJ-001	X	-	X
OBJ-003	X	X	X
OBJ-007	-	-	G

where: A, B, and G denote `satellite_A`, `satellite_B`, and `ground_station`, respectively.

Task P2: Missing Data Detection

Create a script:

```
detect_missing_objects.sh
```

The script should:

- identify objects that do not appear in all providers,
- classify such objects as incomplete across the data space,
- display which providers are missing for each incomplete object.

Assume that the complete set of providers is known (e.g. `satellite_A`, `satellite_B`, `ground_station`), and we should get something like:

```
OBJ-001 missing in: satellite_B
OBJ-007 missing in: satellite_A, satellite_B
```

Task P3: Federated Query Risk Analysis

Create a script:

```
federated_risk_analysis.sh
```

The script should:

- compare full vs federated query results,
- calculate how many observations are missing from the federated result compared with the full result,
- compute the percentage of missing data relative to the full query result.

Assume that the federated query uses only a selected subset of providers (e.g. `satellite_A` and `satellite_B`), while the full query uses all providers in the data space.

and, all in all we should get something like:

```
FULL: 15
FEDERATED: 10

MISSING: 5
LOSS: 33.3%
```

Task P4: Cross-Provider Value Consistency

Create a script:

```
value_consistency_check.sh
```

The script should:

- compare selected numerical attributes (e.g. `temperature`, `velocity`) for the same object across providers,
- perform comparisons only for objects that appear in more than one provider,
- detect significant differences between providers (e.g. differences above a selected threshold),
- highlight inconsistent observations.

And we should get something similar to:

```
OBJ-003 inconsistency detected (temperature):

satellite_A: 15.0
satellite_B: 14.8
ground_station: 15.2
```

This indicates potential measurement differences or data quality issues across providers.

Task P5: Dominant Provider Detection

Create a script:

```
dominant_provider.sh
```

The script should:

- count observations per provider,
- display providers ordered from highest to lowest number of observations,
- determine which provider contributes the largest number of observations,
- calculate percentage contribution of each provider relative to the total number of observations.

After completing we should get something similar to:

```
satellite_A: 6 (40%)
satellite_B: 5 (33%)
ground_station: 4 (27%)

DOMINANT PROVIDER: satellite_A
```

Task P6: Data Space Coverage Score

Create a script:

```
data_space_coverage.sh
```

The script should:

- calculate how many objects appear in all providers,
- calculate how many objects are partial, i.e. missing from at least one provider,
- compute a coverage score defined as the percentage of objects that appear in all providers.

Assume that the complete set of providers is known (e.g. `satellite_A`, `satellite_B`, `ground_station`) and we should get something like:

```
TOTAL OBJECTS: 6
FULL COVERAGE: 3
PARTIAL: 3

COVERAGE SCORE: 50%
```

Task P7: Schema Drift Detector

Create a script:

```
scripts/schema_drift_detector.sh
```

The script should:

- compare schemas across providers,
- detect differences,
- report schema drift.

Expected Output

```
SCHEMA STATUS: CONSISTENT
```

or:

```
SCHEMA DRIFT DETECTED:
ground_station has extra column: altitude
```

Task P8: Federated Completeness Checker

Create a script:

```
scripts/federated_completeness.sh
```

The script should:

- simulate a federated query,
- compare with full query,
- determine whether results are complete.

Expected Output

```
FEDERATED RESULT: INCOMPLETE
MISSING PROVIDERS: ground_station
```

Task P9: Data Space Health Indicator

Create a script:

```
scripts/data_space_health.sh
```

The script should:

- combine:
 - coverage,
 - schema consistency,
 - federated completeness,
- classify data space state as:
 - GOOD
 - WARNING
 - CRITICAL

Expected Output

```
DATA SPACE HEALTH: WARNING
Reason: incomplete coverage and federated loss detected
```

14 Submission

Submit your work using both a version control repository and an archive uploaded to the UPEL platform.

Both submissions are required for the laboratory to be considered complete.

Repository Submission

Your repository must include:

- all scripts created during the laboratory, stored in the `scripts` directory,
- the generated report file `reports/query_report.txt`.

After completing the laboratory:

- commit your changes,
- push them to your repository,
- create a **Merge Request** (GitLab) or **Pull Request** (GitHub).

The request must clearly indicate that it contains the solution for:

```
Laboratory 4: Querying and Integrating Data in a Data Space
```

UPEL Submission

In addition to the repository submission, upload a compressed archive to the UPEL platform.

The archive must contain the complete project workspace:

```
oods-system/
  providers/
  scripts/
  reports/
```

Requirements:

- all scripts must be included,

- the file `reports/query_report.txt` must be present,
- the directory structure must be preserved,
- the archive must be named:

```
lab04_oods_<surname>.tar.gz
```

Report File Requirement

The file:

```
reports/query_report.txt
```

must contain:

- results of data space queries,
- object analysis (e.g. OBJ-003),
- provider distribution,
- federated vs full query comparison,
- schema consistency result.

Final Note

Both submissions (repository and UPEL archive) must be complete.

Missing scripts, missing reports, or incorrect structure will result in the laboratory being marked as incomplete.